

ARM NEON汇编相关

目录

- **ARM概念梳理**
 - ARM architecture（架构/体系结构）
 - ARM core
 - ARM CPU（MCU）
 - ARM Soc
- **ARMv7-A && ARMv8-A架构简介**
 - AArch64 Exception level
 - ARMv7之前的实现
 - ARMv7-A的实现
 - ARMv8-A的实现
- **SIMD与NEON概览**
 - ARM SIMD instructions
 - NEON架构（数据类型/寄存器/指令）
 - NEON支持的数据结构
 - NEON寄存器
 - NEON指令
- **NEON汇编指令**
 - AArch64
 - ARMv8 NEON寄存器
 - Floating-point register organization in AArch64
 - Scalar register sizes
 - Vector register sizes
 - NEON汇编指令格式
 - AArch32 / Armv7
 - AArch64 & AArch32& ARMv7-A NEON指令比较
- **常用NEON汇编指令**
 - 内联汇编一般格式
 - 预取指令
 - 访存指令

- 浮点计算指令
- 排列指令
 - 提取指令
 - 转置指令
- 相关资料
 - ARM官网文档
 - 其它参考资料
 - NEON优化经验【持续更新】

ARM概念梳理

ARM architecture（架构/体系结构）

ARM architecture，是指ARM公司开发的、基于精简指令集架构（RISC, Reduced Instruction Set Computing architecture）设计的包括异常模式，寄存器集，内存管理等功能的集合。

- ① ARM体系结构定义了抽象机器的行为，这些行为被称为**处理元素（processing element）**，缩写为PE，与ARM体系结构兼容的实现必须符合PE所描述的行为，一个架构可以定义多种行为。

- ① 执行状态定义了PE的执行环境：

- 支持的寄存器宽度
- 支持的指令集
- 异常模型
- 虚拟内存系统架构
- 编程模型
- ...

- ① 通用名称AArch64和AArch32分别代表了64位和32位执行状态。

AArch64是64位执行状态，这意味着地址保存在64位寄存器中，基本指令集中的指令可以使用64位寄存器进行处理。AArch64支持A64指令集。

AArch32是32位执行状态，这意味着地址保存在32位寄存器中，并且基本指令集使用32位寄存器进行处理。AArch32支持T32（可变速长指令集，同时使用16位和32位指令编码）和A32指令集。

ARM体系结构到目前为止，已经有9个主要版本，由版本号1到9表示。其中前三个版本现在已经过时了，ARMv8向后兼容。日常开发中常见的有ARMv7-A、ARMv8-A、ARMv8.2-A、ARMv9-A。

ARM core

ARM core是基于ARM architecture开发出来的IP core，它是介于architecture和最终的CPU(MCU)之间的中间产品，这也是ARM商业模式的独特之处。从ARMv7开始，ARM的市场开始扩展到移动设备之外的其它领域，这也是ARMv7划分为A（Application）、R（Real-time）和M（Microcontroller）三个系列的原因，其实质就是三个细分市场，其中的A系列，就是针对性能要求较高的应用，也是日常中接触更多的。

ARM core有两种类型：一种是ARM公司自己发布的，如我们耳熟能详的ARM Cortex A77、ARM Cortex X2等等；另一种是ARM授权其它公司开发的Core，如苹果的A13等等。下面是维基百科上的ARM core的列表：https://en.wikipedia.org/wiki/List_of_ARM_processors

ARM CPU (MCU)

其它的芯片厂商，如Phillips、ST、TI等，会基于ARM公司发布的Core，开发自己的ARM处理器，这称作ARM CPU（也可称为MCU）。这些是嵌入式领域中接触最多的芯片。

ARM Soc

对于一些比较专业的应用场景，如视频、音频等，为了追求更小的size、更低的功耗，厂商会在芯片上，集成除处理器之外的东西，如视频编解码器、DSP、NPU等。这些集成了其它功能的芯片，称作片上系统（SOC），如华为的麒麟芯片。

ARMv7-A && ARMv8-A架构简介

由于处理器性能越来越强，运行于其上的软件也越来越复杂，复杂到单一应用对内存的需求可能超出32-bit架构所能支持的最大内存（4G），但受限于32-bit的指令集，虚拟地址寻址空间只有32bits（4G），如果有应用需要更大的虚拟内存，怎么办？只能定义一个新的架构，即我们常说的ARM64。

当然，ARMv8并不仅仅是为了解决虚拟地址的问题，它也要解决ARMv7之前架构的一些历史问题。

ARMv8-A架构的主要特性包括：

1. 新增一套在AArch64执行状态下工作的指令集，称作A64。
2. 由于需要向前兼容ARMv7，所以同时支持现存的32-bit指令集，称作A32和T32（即ARM和Thumb指令集）。
3. 定义AArch64和AArch32两套运行状态（称作Execution state），分别执行64-bit和32-bit指令集。软件可以在需要的时候，切换Execution state。
4. AArch64最大的改动，是使用新的概念（exception level），整合了之前的processor mode、privilege level等概念。
5. 在ARMv7 security extension的基础上，新增security model，支持安全相关的应用需求。

6. 在ARMv7 virtualization extension的基础上，提供完整的virtualization框架，从硬件上支持虚拟化。

AArch64 Exception level

Exception level，是ARMv8-A引入的一个新概念，用于整合之前架构中processor mode和privilege level相关的功能。

ARMv7之前的实现

以前的ARM架构，处理器可以工作在多种模式下（称作processor mode），包括User、FIQ、IRQ、Abort、Undefined、System等。

i 之所以存在不同的模式，主要有2个方面的考虑：

1. 不同的处理器模式，有不同的硬件访问权限，称作privilege level。主要有2个level，privilege和non-privilege。其中只有User模式属于non-privilege level，其它均是privilege level。安全起见，大多数时候，软件都运行在User mode。一旦需要其它操作，则需要切换到相应的privilege模式下。
2. 这些处理器模式，除User模式外，其它模式基本上和各类异常一一对应。而不同的模式，都有一些自己独有的寄存器，例如R13(SP)、R14(LR)等等，可以使模式切换过程（也是异常处理过程）更为高效、便利。

ARMv7-A的实现

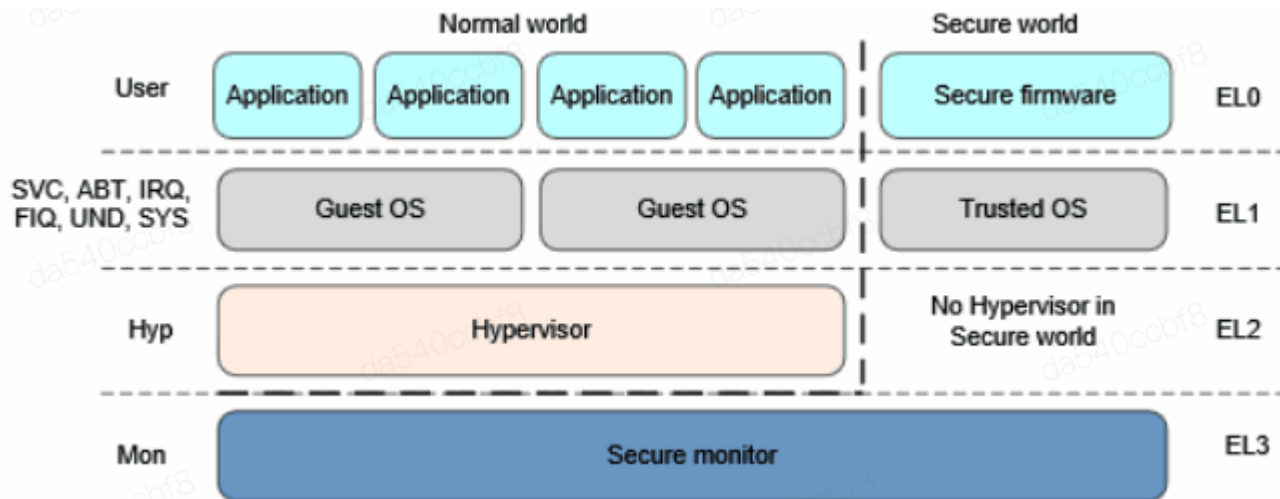
ARMv7-A基本保留了之前的设计，不同之处是将privilege level命名了，称作PL0和PL1，后来出现了PL2，用于虚拟化扩展（Virtualization Extension）。

另外，还增加了两个处理器模式：Monitor和Supervisor，分别用于security扩展和virtualization扩展。

ARMv8-A的实现

ARMv8-A的设计者把processor mode的概念淡化了，取而代之的是4个固定的Exception level，简称EL0-EL3。同时，也淡化了privilege level的概念。Exception level本身就已经包含了privilege的level信息，ELn的privilege level随着n的增大而增大。因此，可以概括地将EL0归属于non-privilege level，EL1/2/3归属于privilege level。

这些Exception level的现实意义如下图所示，先忽略Secure model有关的内容：



- i** 以下是关于Exception level的简单说明：
1. 首先需要注意的是，在AArch64中，已经没有User、SVC、ABT等处理器模式的概念，但由于ARMv8需要向前兼容，所以在AArch32中，就把这些处理器模式map到了这4个Exception level上。
 2. Application位于特权等级最低的EL0，Guest OS（Linux kernel、window等）位于EL1，提供虚拟化支持的Hypervisor位于EL2（为扩展功能，可以不实现），提供Security支持的Security Monitor位于EL3（为扩展功能，可以不实现）。
 3. 只有在异常发生时（或者异常处理返回时），才能切换Exception level（这也是Exception level的命名原因，为了处理异常）。当异常发生时，有两种选择，停留在当前的EL，或者跳转到更高的EL，EL不能降级。同样，异常处理返回时，也有两种选择，停留在当前EL，或者调到更低的EL。

SIMD与NEON概览

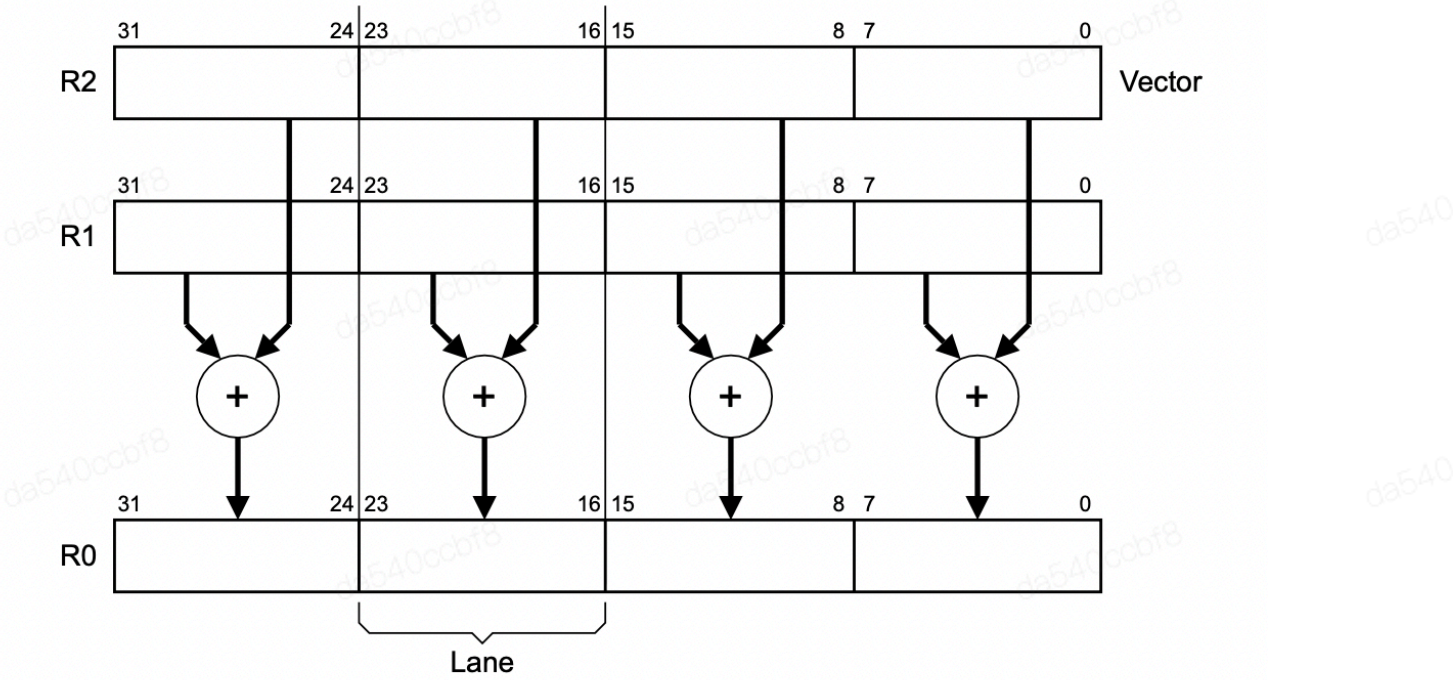
Single Instruction Multiple Data (SIMD)是“一条指令处理多个数据（一般是以2为底的指数数量）”的并行处理技术，相比于“一条指令处理一个数据”，运算速度将会大大提高。

许多程序需要处理大量的数据集，而且很多都是由少于32bits的位数来存储的。比如在视频、图形、图像处理中的8-bit像素数据；音频编码中的16-bit采样数据等。在诸如上述的情形中，很可能充斥着大量简单而重复的运算，且少有控制代码的出现。因此，SIMD就擅长为这类程序提供更高的性能。

ARM SIMD instructions

ARMv6中首次引入了一组规模较小的SIMD指令集，用来处理被pack进32位寄存器中的多个8-bits或16bits数据，这使得对应操作的运算速度提升了2~4倍。

下图显示了指令UADD8 R0、R1、R2所对应的操作。此操作执行四个通道中元素的并行加法。



NEON架构（数据类型/寄存器/指令）

ARMv7架构引入了高级SIMD扩展作为ARMv7-A和ARMv7-R架构的可选扩展。它主要定义了两类寄存器：64位的D寄存器和128位的Q寄存器，以及对这两种寄存器进行操作的指令，这些概念的具体实现称之为NEON技术。NEON技术从ARMv7开始被引入，目前可以在ARM Cortex-A和Cortex-R系列处理器中采用。

接下来，以ARMv7架构为例介绍NEON技术的一些细节。

NEON支持的数据结构

i The NEON instructions support 8-bit, 16-bit, 32-bit, and 64-bit signed and unsigned integers.

NEON also supports 32-bit single-precision floating point elements, and 8-bit and 16-bit polynomials.

The VCVT instruction converts elements between single-precision floating-point and:

- 32-bit integer
- fixed-point
- half-precision floating point, if the processor implements the half-precision extensions

NEON寄存器

NEON寄存器组由32个64位寄存器组成，高级SIMD扩展和VFP单元共享这些寄存器组。如下所示：

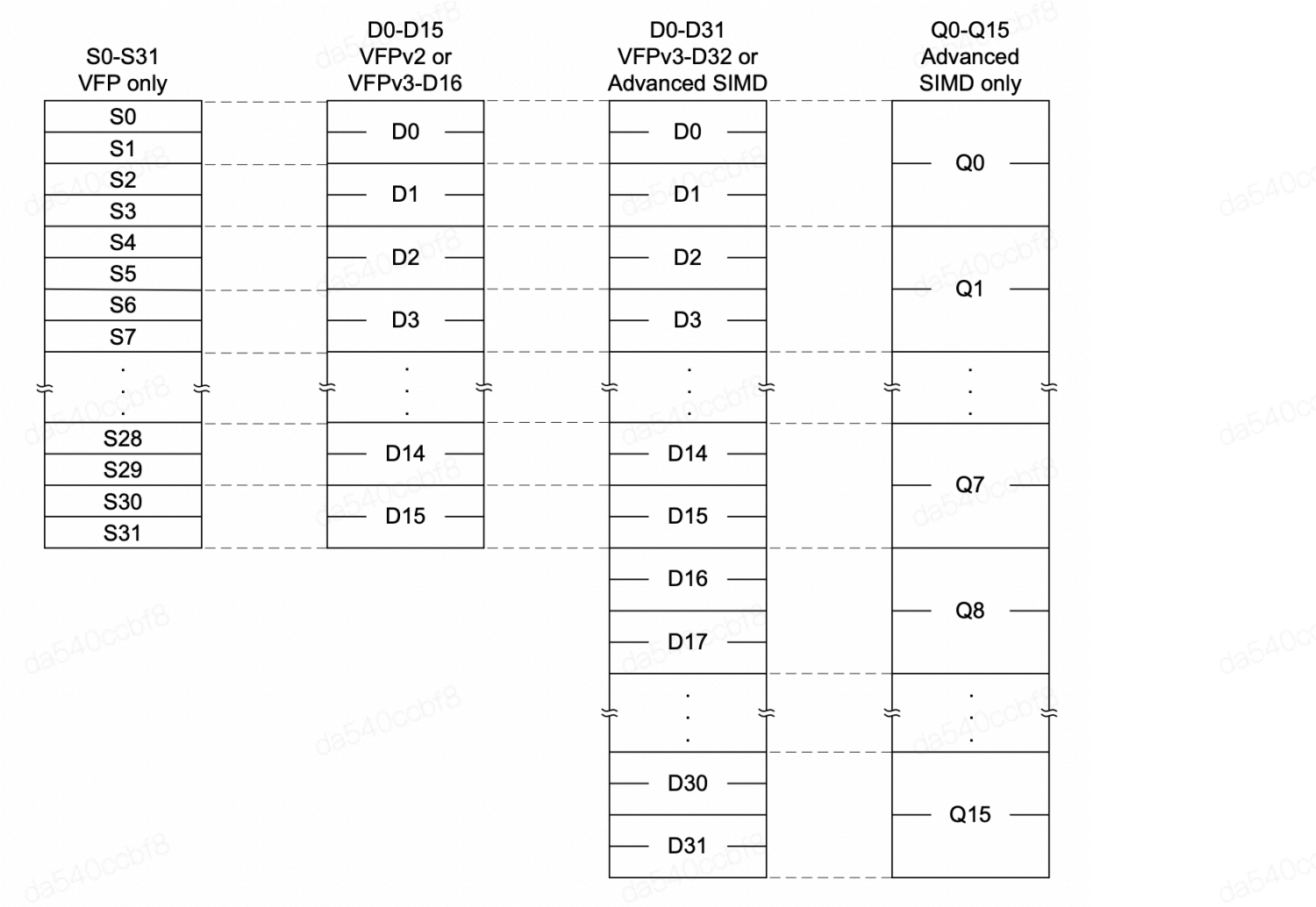


Figure 1-3 NEON and VFP register set

NEON寄存器有以下两种形式：

- sixteen 128-bit quadword registers, Q0-Q15
- thirty-two 64-bit doubleword registers, D0-D31.

每一个Q0-Q15寄存器映射到一对D寄存器。

- D<2n> 映射到Qn的低半部；
- D<2n+1> 映射到Qn的高半部；

NEON指令

按照功能，NEON指令主要包括以下几种类型：

- memory accesses
- data copying between NEON and general purpose registers
- data type conversion
- data processing.

按照操作的数据类型，NEON数据处理指令可以分为以下几种类型：

- **Normal instructions** can operate on any vector types, and produce result vectors the same size, and usually the same type, as the operand vectors.

- **Long instructions** operate on doubleword vector operands and produce a quadword vector result. (操作双字vectors, 生成四倍长字vectors) The result elements are usually twice the width of the operands, and of the same type. (结果的宽度一般比操作数加倍, 同类型) Long instructions are specified using an L appended to the instruction. (在指令中加L)
- **Wide instructions** operate on a doubleword vector operand and a quadword vector operand, producing a quadword vector result. (操作双字 + 四倍长字, 生成四倍长字) The result elements and the first operand are twice the width of the second operand elements. (结果和第一个操作数都是第二个操作数的两倍宽度) Wide instructions have a W appended to the instruction. (在指令中加W)
- **Narrow instructions** operate on quadword vector operands, and produce a doubleword vector result. (操作四倍长字, 生成双字) The result elements are usually half the width of the operand elements. (结果宽度一般是操作数的一半) Narrow instructions are specified using an N appended to the instruction. (在指令中加N)
- **Saturating variants**
 - ARM中的饱和算法:
 - 对于有符号饱和运算, 如果结果小于 -2^n , 则返回的结果将为 -2^n ;
 - 对于无符号饱和运算, 如果整个结果将是负值, 那么返回的结果是 0; 如果结果大于 $2^n - 1$, 则返回的结果将为 $2^n - 1$;
 - NEON中的饱和算法: 通过在V和指令助记符之间使用Q前缀可以指定饱和指令, 原理与上述内容相同。

下图解释了上述指令的操作原理:

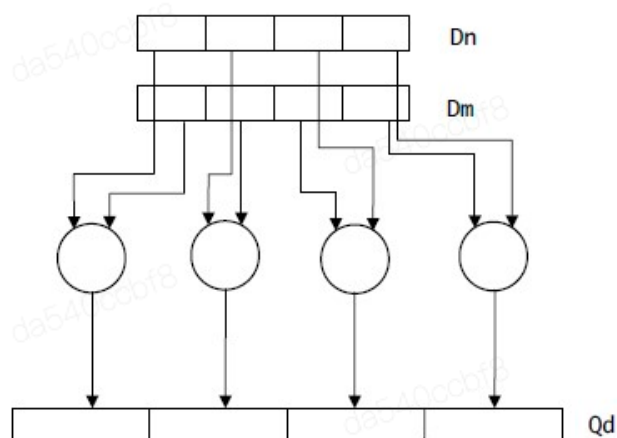


Figure 7-5 NEON long instructions

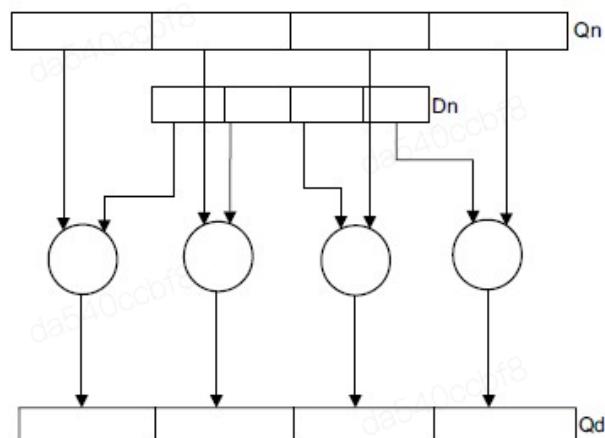


Figure 7-6 NEON wide instructions

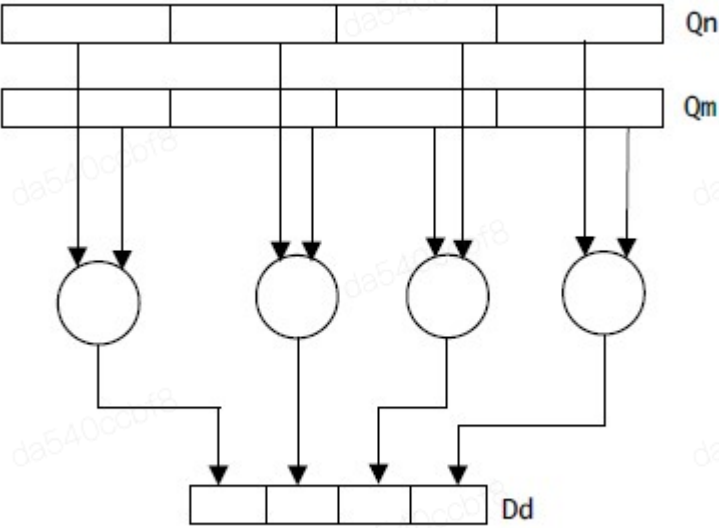


Figure 7-7 NEON narrow instructions

NEON汇编指令

前面已经介绍过，ARMv8-A支持两种执行状态AArch64和AArch32，其中AArch32兼容ARMv7-A。

- AArch64 是 64 位执行状态，支持 A64 指令集。
- AArch32 是 32 位执行状态，支持 T32 和 A32 指令集，同时 AArch32 与 AArch64 中一些的功能保持一致。

 ARMv8支持浮点类型的除法向量操作，这是ARMv7所没有的。另外AArch64还支持double类型的操作。

AArch64与AArch32 / Armv7-A的NEON汇编指令除了种类上存在差异，格式上也存在很大差异。

其中指令中有一些通用的书写格式, 含义如下:

- {}, 表示可选项
- <>, 表示必选项

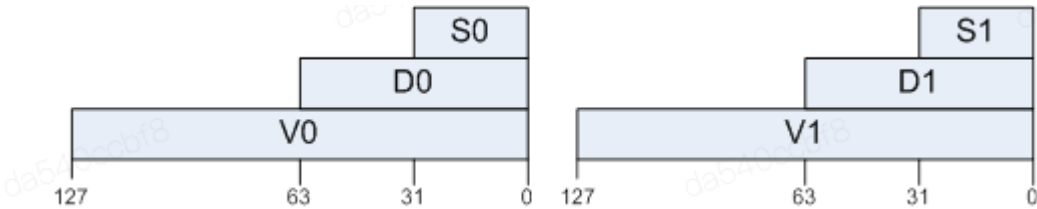
接下来分别对两种执行状态下的NEON汇编指令格式进行介绍。

AArch64

在介绍ARMv8 NEON指令格式之前，首先介绍ARMv8架构的NEON技术相较于AMRv7的一些改变：

ARMv8 NEON寄存器

相对于ARMv7的16个Q（128bits）寄存器，ARMv8的向量寄存器数目翻倍，为32个V寄存器，V0-V31。这些寄存器既可以用于保存标量浮点指令的浮点操作数，还可以用来保存NEON操作的标量和矢量操作数。



接下来，按照不同的类型对这些寄存器的使用进行介绍：

Floating-point register organization in AArch64

当使用通用浮点指令和NEON指令进行标量操作时，寄存器中只有低位会被访问，未使用的高位在读取时被忽略，在写入时被设置为零。

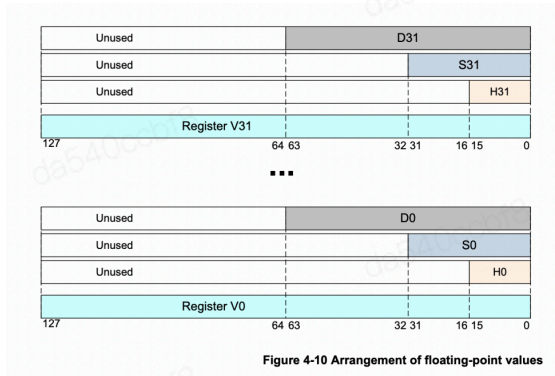


Table 4-7 Operand name for differently sized floats

Precision	Size (bits)	Name
Half	16	Hn
Single	32	Sn
Double	64	Dn

支持16位浮点，但其仅在格式转换时被使用，数据处理类指令并不支持使用16位浮点数。

Scalar register sizes

不同数据size的标量寄存器间的映射关系相较于ARMv7也发生了改变。

- The bottom 64-bits of each of the Q registers can also be viewed as D0-D31, 32 64-bit wide registers for floating-point and NEON use.
- The bottom 32-bits of each of the Q registers can also be viewed as S0-S31, 32 32-bit wide registers for floating-point and NEON use.
- The bottom 16-bits of each of the S registers can also be viewed as H0-H31, 32 16-bit wide registers for floating-point and NEON use.
- The bottom 8-bits of each of the H registers can also be viewed as B0-B31, 32 8-bit wide registers for NEON use.

Only the bottom bits of each register set are used in each case. The rest of the register space is ignored when read, and filled with zeros when written.

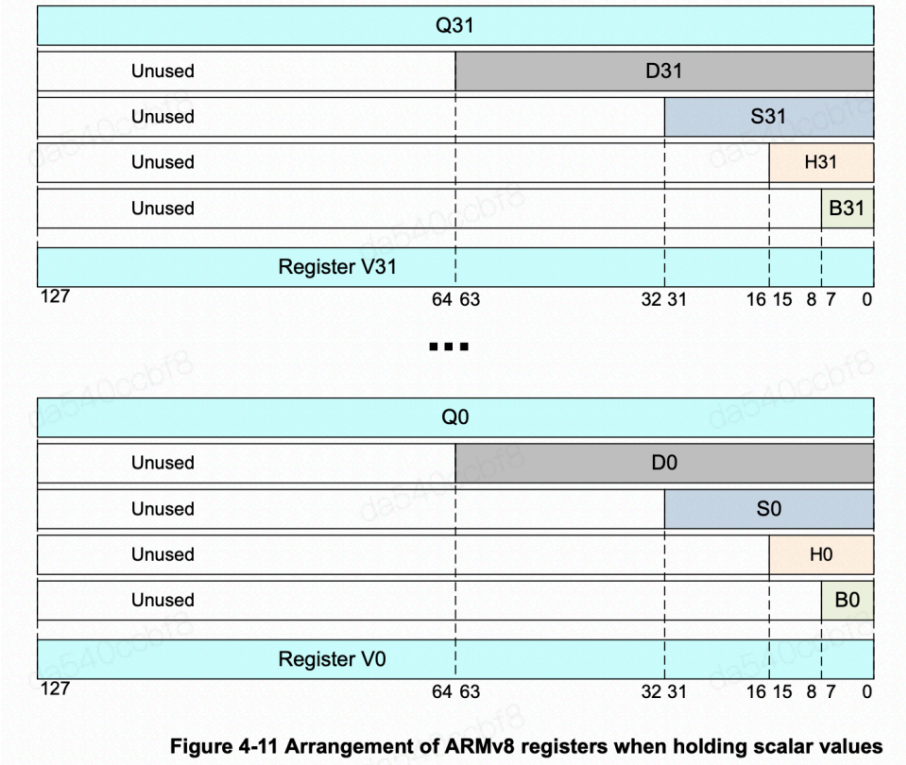


Figure 4-11 Arrangement of ARMv8 registers when holding scalar values

Vector register sizes

矢量寄存器可以是64位宽，带有一个或多个元素，或者128位宽，具有两个或多元素，如下图所示：

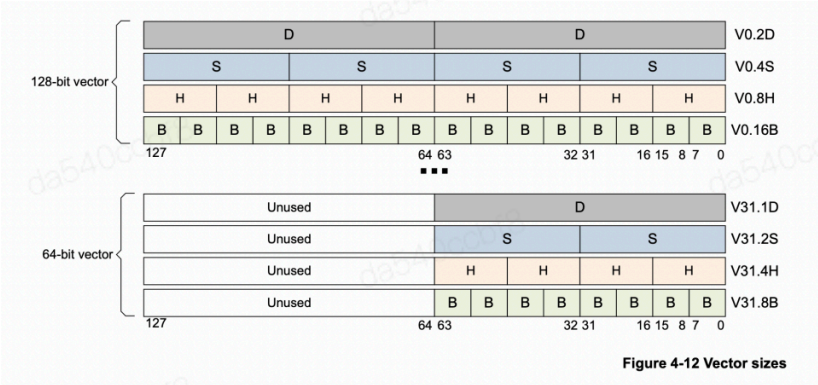


Figure 4-12 Vector sizes

Table 4-9 Operand names for different size vectors

Name	Shape
Vn.8B	8 lanes, each containing an 8-bit element
Vn.16B	16 lanes, each containing an 8-bit element
Vn.4H	4 lanes, each containing a 16-bit element
Vn.8H	8 lanes, each containing a 16-bit element
Vn.2S	2 lanes, each containing a 32-bit element
Vn.4S	4 lanes, each containing a 32-bit element
Vn.1D	1 lane containing a 64-bit element
Vn.2D	2 lanes, each containing a 64-bit element

NEON汇编指令格式

i {<prefix>}<op>{<suffix>} Vd.<T>, Vn.<T>, Vm.<T>

- **<prefix>** Represents data type
 - **S/U/F/P**：表示数据类型，分别为 有符号整型/无符号整型/浮点型/布尔型。
 - **Q**：表示饱和（Saturating）计算。
 - **R**：表示舍入（Rounding）计算, Rounding 操作等价于加上 0.5 之后再截断。
 - **H**：表示折半（Halving）计算。
 - **D**：表示翻倍（Doubling）算。
- **<op>** Instruction mnemonic(for example, ADD, SUB, MUL).
- **<suffix>** For special purpose functions.

- **V**: 表示 Reduction 计算。
- **P**: 表示 Pairwise 计算。e.g faddp
- **H**: 表示结果只取每个通道的高半部分 (High)。
- **L/N/W/L2/N2/W2**: 表示数据处理指令的类型及效果。
 - **L / L2**: 表示输出向量是输入向量长度的 2 倍, 其中 L 表示输入寄存器的低 64bit 数据有效, L2 表示输入寄存器的高 64bit 数据有效。
 - **N/N2**: 表示输出向量是输入向量的 1/2 倍, N 表示输出向量只有低 64bit 有效, N2 则表示输出向量只有高 64bit 有效。
 - **W/W2**: 表示输出向量和第一个输入向量长度相等, 且这两个向量是第二个向量长度的 2 倍, 其中 W 表示第二个输入向量的低 64bit 有效, W2 表示第二输入向量的高 64bit 有效。
- **<T>** Packing format. [8B, 16B, 4H, 8H, 2S, 4S, 2D]. B=byte, H=halfword(16-bit), S=word (32-bit), D=doubledword (64-bit)

AArch32 / Armv7

所有支持的AArch32 NEON指令都有一个助记符 **V**, 下面以32位指令为例, 它们的一般格式如下所示:

V{<mod>}<op>{<shape>}{<cond>}{.<dt>} <dest1>{, <dest2>}, <src1>{, <src2>}

- **<mod>**
 - **Q**: The instruction uses saturating arithmetic, so that the result is saturated within the range of the specified data type, such as **VQABS**, **VQSHL** etc.
 - **H**: The instruction will halve the result. It does this by shifting right by one place (effectively a divide by two with truncation), such as **VHADD**, **VHSUB**.
 - **D**: The instruction doubles the result, such as **VQDMULL**, **VQDMLAL**, **VQDMLSL** and **VQ{R}DMULH**.
 - **R**: The instruction will perform rounding on the result, equivalent to adding 0.5 to the result before truncating, such as **VRHADD**, **VRSHR**.
- **<op>** - the operation (for example, **ADD**, **SUB**, **MUL**).
- **<shape>** - Shape, 即前文中的Long (L), Wide (W), Narrow (N).
- **<cond>** - Condition, used with IT instruction.
- **<.dt>** - Data type, such as s8, u8, f32 etc.
- **<src1>, <src2>** Source registers. Some instructions have an immediate value operand.
- **<dest1>, <dest2>** Destination registers.

AArch64 & AArch32 & ARMv7-A NEON指令比较

此处仅列出对比表格, 更多细节可参考官网社区博客[arm-commarm-neon-programming-quick-reference](https://community.arm.com/developer/quick-starts/quick-reference)

下表比较了ARMv7-A、AArch32和AArch64 NEON指令集。

“√”表示AArch32 NEON指令的格式与ARMv7-A NEON指令相同。

“Y”表示AArch64 NEON指令具有与ARMv7-A NEON指令相同的功能，但格式不同。需要查看ARMv8-A的架构文档。

1		ARMv7-A	AArch32
2	logical and compare	VAND, VBIC, VEOR, VORN, and VORR (register)	√
3		VBIC and VORR (immediate)	√
4		VBIF, VBIT, and VBSL	√
5		VCEQ, VCGE, VCGT, VCLE, and VCLT	√
6			
6	general data processing	VCVT (between fixed-point or integer, and floating-point)	√
7		VCVT (between half-precision and single-precision floating-point)	√
8		n/a	n/a
9		VDUP	√
10		VEXT	√
11		VREV	√
12		VUZP, VZIP	√
13		VTRN	√
14	shift	VSHL, VQSHL, VQSHLU, and VSHLL (by immediate)	√
15		VQ{R}SHR{U}N	√
16	general arithmetic	VMAX, VMIN, VPMAX, and VPMIN	√
17		V{Q}ADD, VADDL, VADDW, V{Q}SUB, VSUBL, and VSUBW	√
18		VPADD{L}, VPADAL	√
19		VCLS, VCLZ, and VCNT	√
20	multiply	VMUL{L}, VMLA{L}, and VMLS{L}	√

21		VMUL{L}, VMLA{L}, and VMLS{L} (by scalar)	√
22		VFMA, VFMS	√
23		VQDMULL, VQDMLAL, and VQDMLSL (by vector or by scalar)	√
24		VQ{R}DMULH (by vector or by scalar)	√
25		n/a	n/a
26		n/a	n/a
27	load and store	VLDn/VSTn(n=1, 2, 3, 4)	√
28		VPUSH/VPOP	√

常用NEON汇编指令

由于Lite支持的ARM Soc众多，支持的架构范围大（从ARMv7到ARMv9-A），因此在日常开发中，往往需要考虑不同ARM core在不同执行状态下的兼容性问题，通常表现在同一份内联汇编优化代码需要实现用__aarch64__宏隔开的两份代码（分别跑在AArch64和AArch32下），如下所示：

```
> #ifdef __aarch64__ ...
> #else ...
#endif // __aarch64__
```

NEON指令按照作用可以分为：logical and compare、general data processing、shift、general arithmetic、multiply、load and store等。接下来介绍日常开发中常用到的一些汇编指令，以AArch64格式为例。

内联汇编一般格式

当前Lite中主要是通过内联汇编的形式来进行深度优化，因此本小节先介绍一下ARM内联汇编的一般格式，[内联汇编-gcc](#)给出了更详细的说明。

</> 内联汇编一般格式 C | 收起 ^

```

1 __asm__ qualifiers (
2
3     // 汇编代码部分
4
5     : OutputOperands //在内联汇编代码中被修改的变量列表
6     : InputOperands  //在内联汇编代码中用到的变量列表
7     : Clobbers        //在内联汇编代码中用到的寄存器列表
8 );

```

- **qualifiers** : 一般是用 **volatile** 修饰词
- **OutputOperands** : 在内联汇编中会被修改的变量列表, 变量之间用','隔开,
 - 每个变量的格式是: `[asmSymbolicName] "constraint"(cvariablename)`
 - **cvariablename** : 表示变量原来的名字;
 - **asmSymbolicName** : 表示变量在内联汇编代码中的别名, 一般和cvariablename一样, 在汇编代码中就可以通过 `%[asmSymbolicName]` 去使用该变量;
 - **constraint** :一般填 `=r`, 具体解释见gcc的内联汇编文档。
- **InputOperands** : 在内联汇编中用到的所有变量列表, 变量之间用','隔开,
 - 每个变量的格式是: `[asmSymbolicName] "constraint"(cexpression)`
 - 和输出不一样的是, 首先要按 **OutputOperands** 列表的顺序再列一遍, 但是 **constraint** 用数字代替从 `0` 开始, 然后才是写其他只读变量, 只读变量 **constraint** 填 `r`。
- **Clobbers** : 一般是 `"cc"`, `"memory"` 开头, 然后接着填内联汇编中用到的通用寄存器和向量寄存器
 - `"cc"` 表示内联汇编代码修改了标志寄存器;
 - `"memory"` 表示汇编代码对输入和输出操作数执行内存读取或写入操作 (读写参数列表之一的变量指向的内存);

预取指令

The instruction syntax is:

PRFM <prfop>, <addr> | label

Where prfop is a concatenation of the following options:

Type	PLD or PST (prefetch for load or store).
Target	L1, L2, or L3 (which cache to target).
Policy	KEEP or STRM (keep in cache, or streaming data).

For example, `PLDL1KEEP`.

 `prfm pldl1keep, [x0, #256]`

使用经验

- 不写 offset 不如写个 #128
- load 128bit用#128, 256bit或更多用#256
- KEEP适合连续读, STRM适合提前准备要跳到很远的地方读, 比如换channel
- 读很多数据, 用不同offset连续两次pldl2strm

访存指令

NEON的访存指令和通常意义上的SIMD有较大区别, 原因在于ARM64没有一个完整的新指令集来执行SIMD操作, 比如并行乘法, 而是使用了许多与浮点标量代码相同的指令, 但是通过控制操作数的类型, 因此这些指令可以被识别并作为SIMD单元运行。

例如, 可以用如下指令将两个双精度浮点标量相乘:

</> 浮点标量乘

C | 收起 ^

```
1 FMUL D0, D1, D2
```

而同样的指令还可以用于执行四个浮点数的SIMD操作:

</> 浮点向量乘

C | 收起 ^

```
1 FMUL V0.4S, V1.4S, V2.4S
```

通过这种指定向量寄存器中每个操作数长度 (lane) 的方式, 汇编器可以识别同一条指令表示的是标量还是向量操作, 而向量寄存器的使用和表示方式也更加灵活。

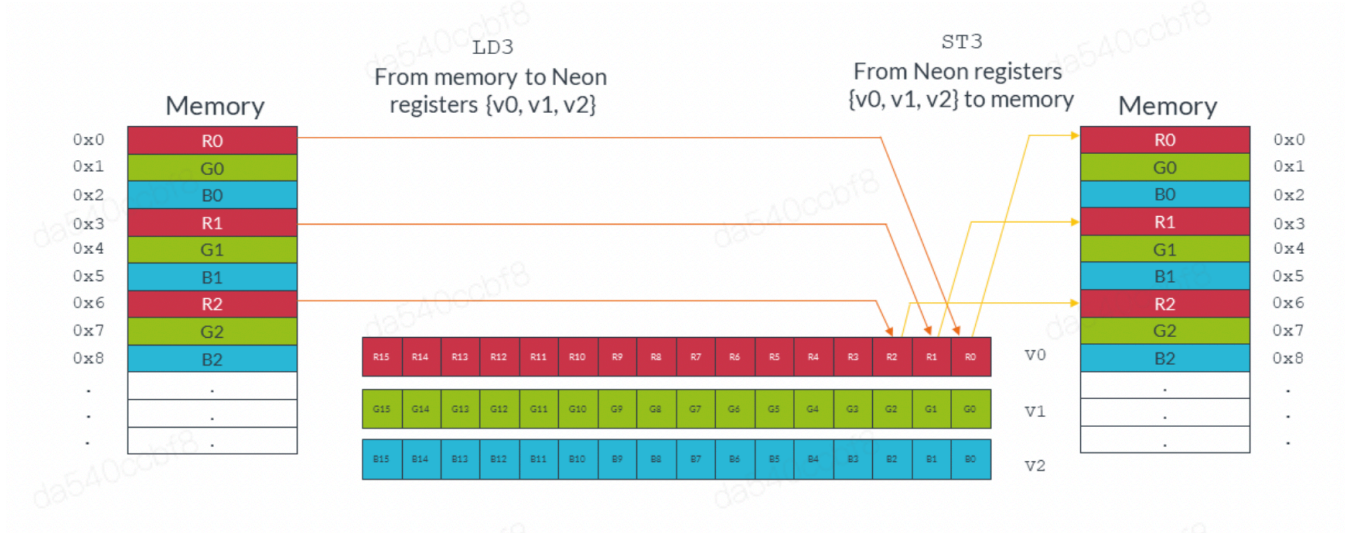
对于访存而言, NEON有一个特殊的Interleave pattern。

i Neon provides instructions to load and store interleaved structures containing from one to four equally sized elements. Elements are the standard Neon-supported widths of 8 (B), 16 (H), 32 (S), or 64 (D) bits.

- LD1 is the simplest form. It loads one to four registers of data from memory, with no deinterleaving. You can use LD1 to process an array of non-interleaved data.
- LD2 loads two or four registers of data, deinterleaving even and odd elements into those registers. You can use LD2 to separate stereo audio data into left and right channels.
- LD3 loads three registers and deinterleaves. You can use LD3 to split RGB pixel data into separate color channels.
- LD4 loads four registers and deinterleaves. You can use LD4 to process ARGB image data.

The store instructions ST1, ST2, ST3, and ST4 support the same options, but interleave the data from registers before writing them to memory.

下图展示了RGB数据的结构化读取和存储的具体实现过程：



浮点计算指令

fmla指令常见于GEMM相关的汇编代码中，它是为了加速乘加操作而设计出的特殊指令。

Floating-point fused multiply-add to accumulator.

Syntax

FMLA *Vd.T, Vn.T, Vm.T*

Where:

Vd

Is the name of the SIMD and FP destination register.

T

Is an arrangement specifier, and can be one of 2S, 4S or 2D.

Vn

Is the name of the first SIMD and FP source register.

Vm

Is the name of the second SIMD and FP source register.

通常有以下几种用法：

- 两个浮点标量相乘，并与目标浮点寄存器相加

FMLA <Hd>, <Hn>, <Vm>.*H*[<index>]

- 向量中每个通道中的数据分别与单个元素相乘，并与目标向量相加

FMLA <Vd>.<T>, <Vn>.<T>, <Vm>.<Ts>[<index>]

- 向量与向量逐个元素相乘，并与目标向量相加

FMLA <Vd>.<T>, <Vn>.<T>, <Vm>.<T>

排列指令

NEON中提供了几种不同类型的排列指令来执行不同的操作，有如下几类：

- move指令
- reverse指令
- extraction指令
- transpose指令
- interleave指令
- lookup table指令

介绍两个Lite中比较常见的排列指令：

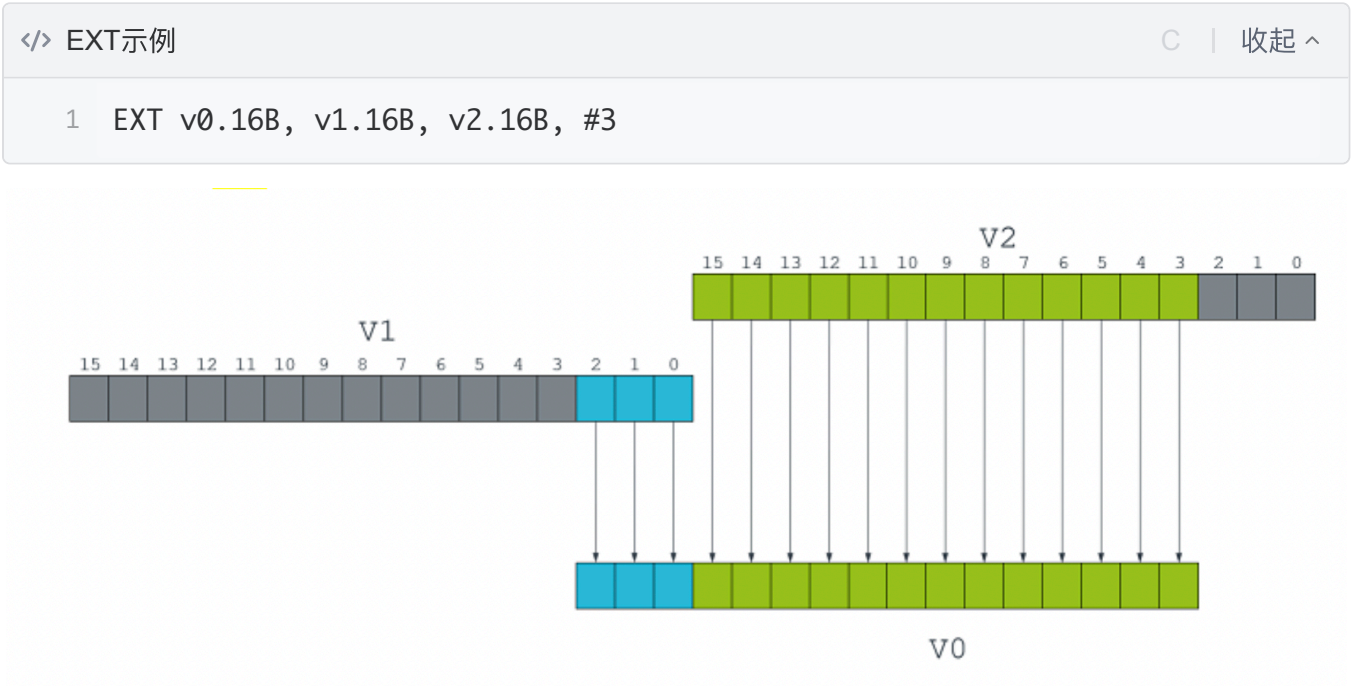
提取指令

提取指令EXT通过从两个不同的源向量中分别提取指定数量的连续通道的数据来创建新向量。

EXT指令通过执行以下操作来构造新向量：

1. 从第二个源向量中，将从第n个通道开始的连续通道的数据复制到目标向量的低位；
2. 从第一个源向量中，将0~n通道的数据填充到目标向量的剩下位置（高位）。

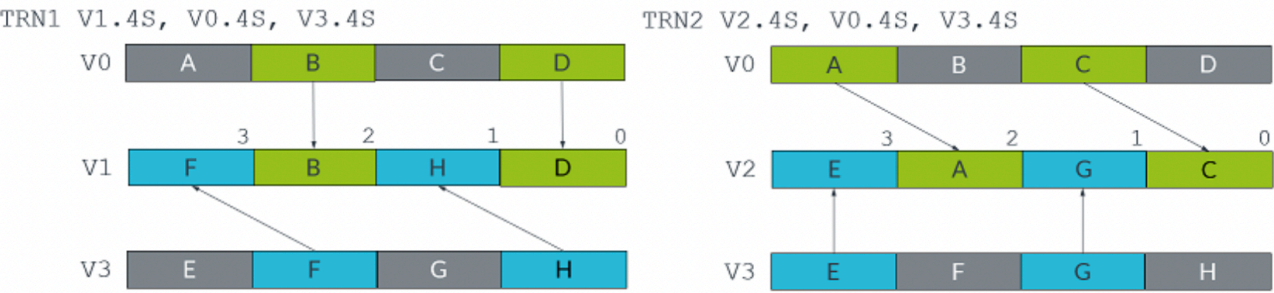
下图展示了示例中指令的具体执行过程：



转置指令

转置指令TRN常用于重排操作，将数据重排提高cache命中率，并且方便批量load数据并使用NEON指令进行优化。与LDn指令不同的是，转置指令interleaves读取来自两个源向量寄存器中的元素。

ARMv8中，NEON提供两个转置指令：TRN1和TRN2。TRN1从两个源向量中交织提取偶数lane处的元素，而TRN2提取奇数lane处的元素。下图展示了这两种指令的具体执行过程：



可以看出：

- TRN1提取Vn、Vm的偶数lane处元素，分别将Vd中提取到的元素放在Vd的偶数lane，将Vm中提取到的元素放在Vd的奇数lane。
- TRN2提取Vn、Vm的奇数lane处元素，分别将Vd中提取到的元素放在Vd的偶数lane，将Vm中提取到的元素放在Vd的奇数lane。

相关资料

ARM官网文档

- 对于ARMv8-A下的NEON汇编，可以参考官网的NEON Programmer Guide进行初步的了解：

<https://developer.arm.com/documentation/102159/0400/?lang=en>

- 对于ARMv7-A下的NEON汇编，官网的NEON Programmer Guide链接如下：

<https://developer.arm.com/documentation/den0018/a/?lang=en>

- 想要了解不同指令集的细节，可以参考：

<https://developer.arm.com/documentation/ddi0597/2022-12/?lang=en>

<https://developer.arm.com/documentation/ddi0596/2021-12/?lang=en>

- 对于更底层的细节，可以先查阅相关ARM core的文档，例如cortex-x2:

<https://developer.arm.com/documentation/101803/0201/?lang=en>

而cortex-x2，又是基于ARMv9-A架构的，想要继续深入了解，可以参考A-profile architecture相关的说明文档：

<https://developer.arm.com/documentation/ddi0487/ia/?lang=en>

其它参考资料

<https://developer.arm.com/documentation#q=NEON>

<https://community.arm.com/arm-community-blogs/b/operating-systems-blog/posts/arm-neon-programming-quick-reference>

http://www.wowotech.net/armv8a_arch/arm_concept.html

http://www.wowotech.net/armv8a_arch/armv8-a_overview.html

https://zyddora.github.io/2016/02/28/neon_1/

<https://aijishu.com/a/1060000000116427#item-1-5>

NEON优化经验【持续更新】

- 每次读入的数据尽可能的占满 128 位
- 除法使用 乘法 进行代替
- 算子进行融合，避免内存的多次读写